50

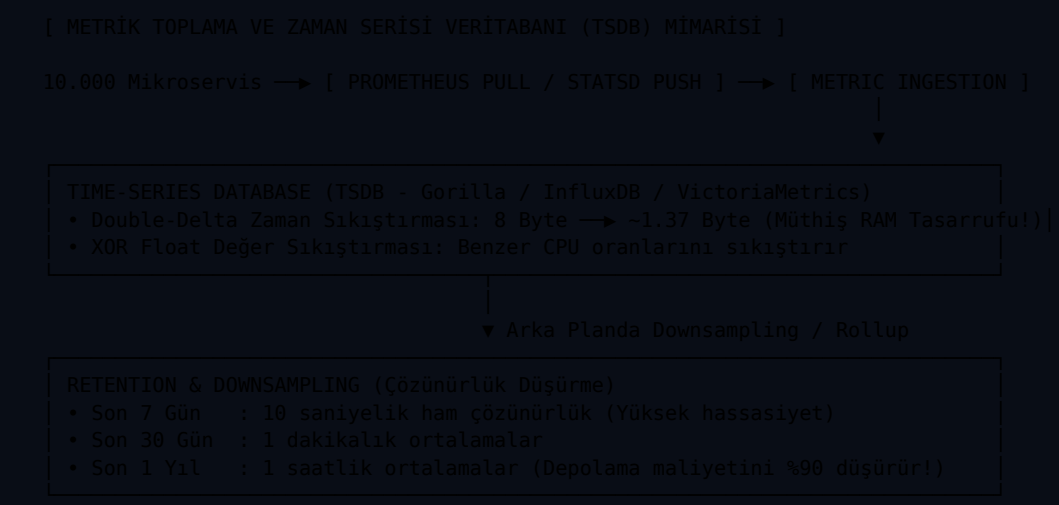
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Metrik ve İzleme Sistemi (Datadog / Prometheus): TSDB ve Downsampling

Designing Metrics & Monitoring (Datadog / Prometheus): TSDB & Aggregations

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir hastanın yoğun bakımda saniyede 1 kez nabzını ölçtüğünüzü düşünün. Bir günde 86.400 nabız verisi çıkar. 1 ay sonra doktor hastanın 3 hafta önceki durumuna bakmak istediğinde saniye saniye 2.5 milyon nabız verisini tek tek okumak istemez; '3 hafta önce her günkü ortalama nabız neydi?' diye sorar! Dağıtık metrik sistemleri de böyledir: Sunucuların CPU kullanımını saniye saniye kaydeder (TSDB); ancak 1 ay geçince o saniyelik verileri silip yerine '1 saatlik ortalama' kaydını bırakır (Downsampling). Böylece hard diskler milyonlarca gereksiz sayıyla dolup taşmaz!

🇬🇧 ENGLISH (ELI5)

Imagine logging an intensive care patient's heart rate every single second. That generates 86,400 numbers a day. Six months later, the doctor reviewing recovery progress doesn't need to scroll through 15 million individual second-by-second data points; they only care about the daily average and peak trends! That is what a Time-Series Monitoring System (Prometheus, Datadog) does: it ingests high-resolution telemetry every 10 seconds, but as data ages past 30 days, background jobs roll up seconds into hourly averages (Downsampling), slashing storage footprint by 90%!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Metrik Sistemi Mimarisi: 1) Pull vs Push Modeli: Prometheus 'Pull' modelini kullanır (Sunucuları kendisi tarar / scrape eder; istemciye yük binmez). Datadog/CloudWatch 'Push' modelini kullanır (Agent metrikleri merkezi API'ye fırlar). 2) Zaman Serisi Veritabanı (TSDB) Sıkıştırması (Facebook Gorilla Mimarisi): Zaman serisi verisi `(Timestamp, Value)` çiftidir. Zaman damgaları 'Double-Delta Compression' ile (ör. her 10 saniyede bir gelen veri için farkın farkı sıfır olur ve tek bir 0 bitiyle temsil edilir); Float değerler ise 'XOR Floating-Point Compression' ile sıkıştırılır. 16 baytlık bir metrik noktası ortalama 1.37 bayta düşürülür (10 kat bellek sıkıştırması!). 3) Downsampling & Rollup: Veri eskidikçe çözünürlük kademeli düşürülür (Raw 10s -> 5m Rollup -> 1h Rollup).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Metrics Systems: 1) Pull vs Push: Prometheus implements a Pull architecture (periodically scrapes `/metrics` endpoints via service discovery; auto-detects dead hosts). Datadog/StatsD uses Push (agents dispatch UDP/TCP packets to gateways; ideal for ephemeral serverless lambdas). 2) TSDB Compression (Facebook's Gorilla Paper): Time-series data comprises `(timestamp, float_value)` streams. Gorilla compresses timestamps via Double-Delta encoding (D = (t[i] - t[i-1]) - (t[i-1] - t[i-2])); periodic intervals evaluate to zero, encoded in a single bit. Float values are compressed by XORing against previous values. Slashes 16-byte tuples down to 1.37 bytes per metric point in memory. 3) Downsampling & Rollups: Background compaction jobs aggregate granular metrics into downsampled buckets (10s raw for 7 days -> 1m rollup for 30 days -> 1h rollup for 1 year), bounding long-term storage growth.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Basitleştirilmiş Zaman Serisi Metrik Veri Noktası ve Rollup Modeli
import java.util.*;

public class MetricsRollupEngine {
    public record MetricPoint(long timestampSeconds, double value) {}

    // Saniyelik verileri 1 dakikalık ortalama noktasına indirgeme (Downsampling)
    public static MetricPoint rollupToMinute(List<MetricPoint> rawPoints) {
        if (rawPoints.isEmpty()) return null;
        double sum = 0;
        for (var p : rawPoints) sum += p.value();
        double avg = sum / rawPoints.size();
        return new MetricPoint(rawPoints.getFirst().timestampSeconds(), avg);
    }
}

void main() {
    // 1 dakikalık sürede toplanan 3 saniyelik CPU verisi: %40, %60, %50
    var raw = List.of(
        new MetricsRollupEngine.MetricPoint(1700000000L, 40.0),
        new MetricsRollupEngine.MetricPoint(1700000010L, 60.0),
        new MetricsRollupEngine.MetricPoint(1700000020L, 50.0)
    );

    var rolledUp = MetricsRollupEngine.rollupToMinute(raw);
    IO.println("Ham Veri Noktası Sayısı: " + raw.size());
    IO.println("Downsampled 1-Dk Ortalama: %" + rolledUp.value() + " [Zaman: " +
rolledUp.timestampSeconds() + "]");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta metrikleri standart ilişkisel veritabanına (MySQL/PostgreSQL) kaydedeceğinizi söylemeyin! Milyonlarca metrik satırı B-Tree indekslerini patlatır ve disk doldurur. Metrikler için daima TSDB (Prometheus, TimescaleDB, VictoriaMetrics, ClickHouse) önerilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch storing high-velocity time-series metrics inside a traditional relational database (MySQL/PostgreSQL). The volume of writes will shred B-Tree indexes and consume petabytes of disk. Prescribe purpose-built TSDBs like VictoriaMetrics or ClickHouse.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- TSDB veritabanları zaman serisi verisini Gorilla double-delta algoritmasıyla 10 kat sıkıştırır.
- Prometheus Pull modeli sunucu keşfiyle hafif çalışırken, Push modeli sunucusuz ortamlar için uygundur.
- Downsampling politikaları eskiyen verileri özetleyerek disk maliyetlerini %90 düşürür.

🎯 Key Takeaways for the Interview

- TSDBs compress timestamp-value streams by 10x using Gorilla double-delta and XOR algorithms.
- Pull architecture suits stateful microservice discovery; Push fits ephemeral serverless functions.
- Downsampling policies aggregate aging historical metrics, bounding storage costs sustainably.